TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI

Thiết kế giao diện đa màn hình: Responsive & Adaptive UI trong Flutter

Sinh viên thực hiện: **Phan Văn Trà**
Trần Đăng Duy

Giảng viên hướng dẫn: **TS. Nguyễn Duy Nhật Viễn**

Đà Nẵng, ngày 26 tháng 11 năm 2026

Mục lục

1. Cơ sở lý thuyết
- 1.1 Khái niệm Responsive và Adaptive
- 1.2 MediaQuery, LayoutBuilder và breakpoint
- 1.3 Nguyên tắc thiết kế cho nhiều màn hình và phương thức nhập liệu

2. Yêu cầu thực hành và demo
- 2.1 Tổng quan project Adaptive Notes
- 2.2 Bố cục thích ứng và minh hoạ mã nguồn
- 2.3 Kịch bản demo và kiểm thử

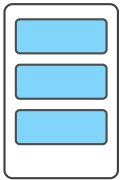
3. Kết luận

1.1 Responsive và Adaptive: hai cách thích ứng giao diện

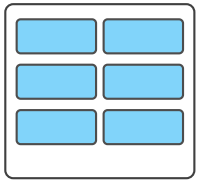
Tiêu chí	Responsive	Adaptive
Ý tưởng	Bố cục **co giãn liên tục** theo không gian khả dụng	**Chọn** một trong nhiều bố cục riêng theo điều kiện
Công cụ	**`Expanded`**, **`Flexible`**, **`GridView`**, **`ConstrainedBox`**	**`LayoutBuilder`** / **`MediaQuery`** + **breakpoint**, trả về widget khác nhau
Ví dụ	Lưới thẻ tự chia **1** → **2** → **3 cột**	Danh sách (mobile) ↔ **Master–Detail** (tablet)
Phù hợp khi	Kích thước thay đổi liên tục (kéo cửa sổ, xoay màn hình)	Cấu trúc giao diện hoặc cách tương tác khác nhau giữa các nhóm màn hình

Thực tế thường **kết hợp cả hai**: **adaptive** chọn bố cục khung, **responsive** co giãn nội dung bên trong.

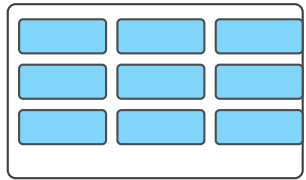
1.1 Minh hoạ trực quan: Responsive vs Adaptive



hẹp: **1 cột**

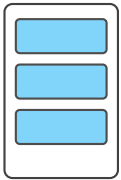


vừa: **2 cột**

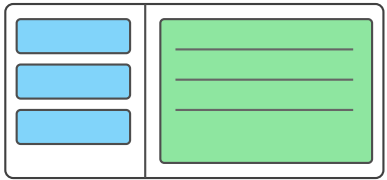


rộng: **3 cột**

Responsive
Cùng một cấu trúc, các phần tử **co giãn / chia lại** khi không gian đổi.



Mobile: danh sách



Tablet: danh sách | chi tiết

Adaptive
Đổi cấu trúc giao diện: 1 vùng ↔ 2 vùng tuỳ điều kiện hiển thị.

1.2 MediaQuery: thông tin về môi trường hiển thị

- **`MediaQuery`** cung cấp thông tin về **cửa sổ / màn hình** của ứng dụng.
- Thuộc tính hay dùng: **`size`**, **`orientation`**, **`padding`** (vùng an toàn), **`textScaler`**, **`platformBrightness`**.
- Dùng hàm chuyên biệt (**`sizeOf`**, **`orientationOf`**, **`textScalerOf`**) để widget **chỉ rebuild** khi đúng thuộc tính đó đổi.
- **Hạn chế**: cho biết kích thước **cửa sổ**, không phải vùng mà widget cha thực sự cấp cho con.

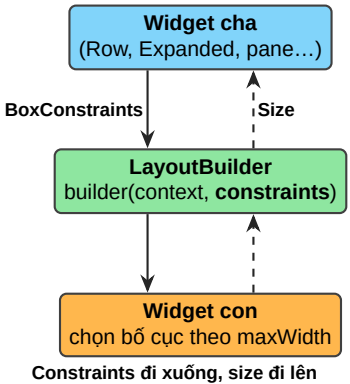
```dart
// Kích thước cửa sổ / màn hình (dp)
final size = MediaQuery.sizeOf(context);
final width = size.width;

// Hướng màn hình
final orientation = MediaQuery.orientationOf(context);
final isLandscape =
    orientation == Orientation.landscape;

// Cỡ chữ người dùng chọn trong hệ thống
final scaler = MediaQuery.textScalerOf(context);

// Vùng an toàn (tai thỏ, thanh trạng thái)
final padding = MediaQuery.paddingOf(context);
```

1.2 LayoutBuilder: đo không gian mà widget cha cấp



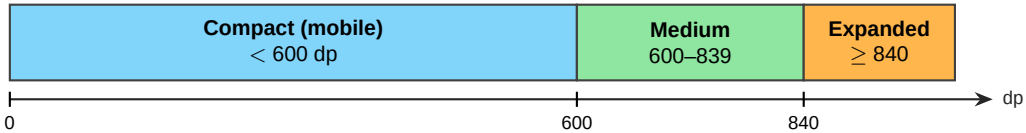

```
LayoutBuilder(
  builder: (context, constraints) {
    // maxWidth: bề rộng tối đa khả dụng cho widget này
    if (constraints.maxWidth >= 600) {
      return const TabletLayout();
    }
    return const MobileLayout();
  },
)
```

- **`builder`** chạy lại **mỗi khi ràng buộc thay đổi**.
- Phản ánh đúng **vùng khả dụng** (pane, split-view, cửa sổ thu nhỏ).

1.2 So sánh MediaQuery và LayoutBuilder

Tiêu chí	MediaQuery	LayoutBuilder
Đo cái gì	**Cửa sổ / màn hình** và thông tin hệ thống	**Ràng buộc** (BoxConstraints) mà widget cha cấp
Dùng khi	Cần cỡ chữ, vùng an toàn, hướng, sáng/tối	Chọn bố cục theo **không gian thực sự khả dụng**
Hạn chế	Không phản ánh vùng nhỏ hơn cửa sổ (pane, split-view)	Không có thông tin hệ thống (cỡ chữ, hướng…)
Khuyến nghị	Dùng cho **thông tin hệ thống**	Dùng để **chọn bố cục**

1.2 Breakpoint: ngưỡng để đổi bố cục



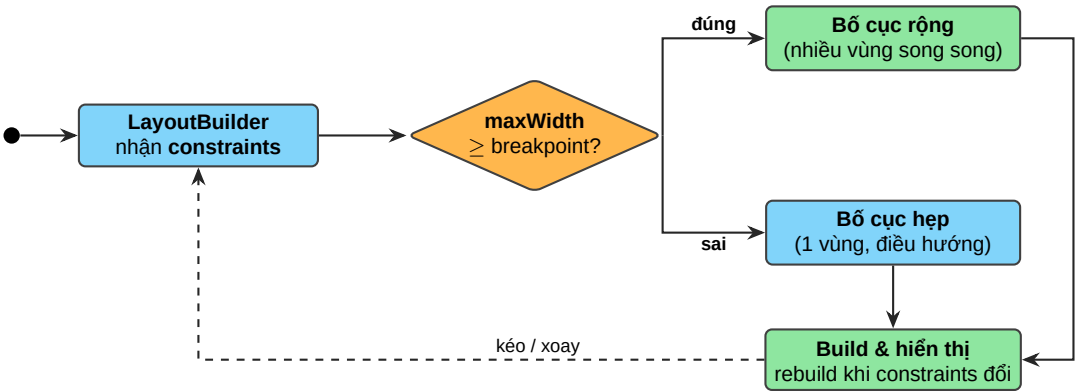
- **Breakpoint** là giá trị bề rộng (đơn vị **dp**) mà tại đó bố cục thay đổi.
- Dựa vào **bề rộng khả dụng**, không dựa vào tên thiết bị.
- Dùng **ít** breakpoint, đặt thành **hằng số** có tên.
- Mốc trên theo phân loại kích thước cửa sổ của **Material 3**.

```
enum ScreenType { mobile, tablet }

class Breakpoints {
  static const double tablet = 600;

  static ScreenType fromWidth(double width) =>
      width >= tablet
          ? ScreenType.tablet
          : ScreenType.mobile;
}
```

1.2 Quy trình chọn bố cục theo không gian khả dụng



1.3 Nguyên tắc thiết kế giao diện thích ứng

1. Dựa vào không gian khả dụng
Đo bằng **LayoutBuilder**, không đoán loại thiết bị hay chỉ dựa hướng xoay.

2. Giới hạn bề rộng nội dung
Dùng **max-width** để dòng chữ dễ đọc, form không bị giãn quá mức.

3. Bảo toàn trạng thái
Đổi kích thước hoặc xoay **không làm mất** lựa chọn và dữ liệu đang xem.

4. Hỗ trợ đa phương thức nhập
Chạm, chuột, bàn phím: mỗi loại có cách tương tác phù hợp.

5. Trợ năng
Tôn trọng **cỡ chữ hệ thống**, theme sáng/tối, vùng chạm đủ lớn.

6. Không để tràn giao diện
Dùng Expanded / Flexible, cuộn nội dung; kiểm thử nhiều kích thước.

1.3 Đa phương thức nhập liệu: Touch, Mouse, Keyboard

Phương thức	Đặc trưng	Cách thích ứng trong Flutter
Touch	Ngón tay, không có hover, cần vùng chạm lớn	Vùng chạm đủ lớn; cử chỉ **vuốt** (`Dismissible`), `GestureDetector`
Mouse	Con trỏ chính xác, có **hover**, chuột phải	`InkWell`, `MouseRegion`, `mouseCursor`; nút bấm **hiển thị sẵn** thay cho cử chỉ ẩn
Keyboard	Phím tắt, di chuyển bằng **focus**	`Shortcuts` / `Actions`, `CallbackShortcuts`, `Focus`

Nguyên tắc: cùng một chức năng nên có **ít nhất một đường thao tác** phù hợp cho mỗi phương thức nhập.

1.3 Tóm tắt phần cơ sở lý thuyết

Responsive / Adaptive

- **Responsive**: co giãn liên tục
- **Adaptive**: chọn bố cục riêng
- Thực tế: **kết hợp** cả hai

Công cụ

- **MediaQuery**: thông tin hệ thống
- **LayoutBuilder**: đo vùng khả dụng
- **Breakpoint**: ngưỡng đổi bố cục

Nguyên tắc

- Dựa vào **không gian khả dụng**
- **Max-width** cho nội dung
- Hỗ trợ **touch, mouse, key-board**
- Bảo toàn **trạng thái**, trợ năng

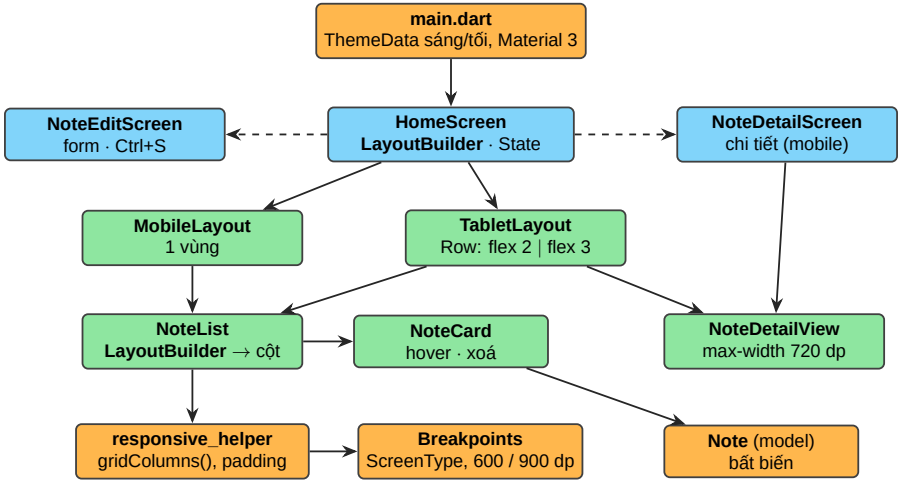
2.1 Adaptive Notes – ứng dụng ghi chú đa màn hình

- Ứng dụng **ghi chú** Flutter minh hoạ **Responsive & Adaptive UI**, **không dùng package bên ngoài**.
- **2 bố cục**: **Mobile** ($<$ 600 dp, 1 vùng) và **Tablet** ($\geq$ 600 dp, **Master–Detail**).
- Lưới thẻ chia **1 / 2 / 3 cột** theo bề rộng thực của vùng danh sách.
- **CRUD** đầy đủ: thêm, sửa, xoá (vuốt hoặc nút), **Hoàn tác** qua SnackBar.
- **Đa phương thức nhập**: chạm, chuột (hover), phím tắt **Ctrl+N**, **Ctrl+S**.
- Theme **sáng / tối**, cỡ chữ hệ thống, thanh **Layout Debug Bar**.

Thành phần	Chi tiết
Ngôn ngữ	Dart
Framework	Flutter, Material 3
Công cụ	`LayoutBuilder`, `MediaQuery`, `Shortcuts`
Dữ liệu	In-memory, 10 ghi chú mẫu
Kiểm thử	`flutter_test`

GitHub: `https://github.com/Duy-OC/Responsive-and-adaptive-UI`

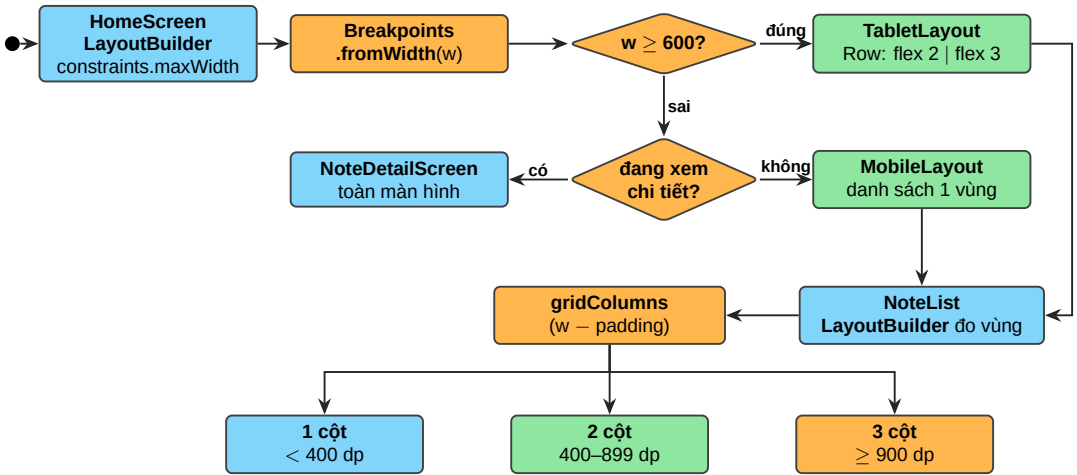
2.1 Kiến trúc project (theo cấu trúc thư mục `lib/`)



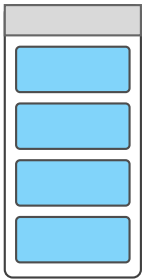
2.2 Đáp ứng yêu cầu thực hành của project

Yêu cầu	Cách project đáp ứng
$\geq$ **2 kích thước / điều kiện hiển thị**	**Mobile** ($<$ 600 dp) và **Tablet** ($\geq$ 600 dp); lưới **1 / 2 / 3 cột**; xoay màn hình; cỡ chữ 1.0–2.0$\times$; sáng/tối
Chọn bố cục theo không gian khả dụng	`HomeScreen`: `LayoutBuilder` đo `maxWidth` $\rightarrow$ `Breakpoints.fromWidth()`. `NoteList`: `LayoutBuilder` riêng đo **bề rộng vùng danh sách** $\rightarrow$ `gridColumns()`
Đa phương thức nhập liệu	**Touch**: chạm, vuốt xoá. **Mouse**: hover, con trỏ `click`, nút xoá. **Keyboard**: **Ctrl+N**, **Ctrl+S**
Trợ năng & chất lượng	`MediaQuery.textScalerOf` co giãn chiều cao thẻ; **max-width** 640 / 720 dp; **Layout Debug Bar**; kiểm thử tự động

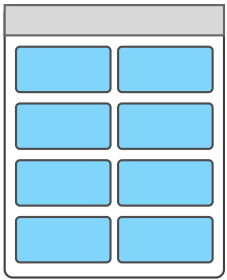
2.2 Sơ đồ quyết định bố cục trong project



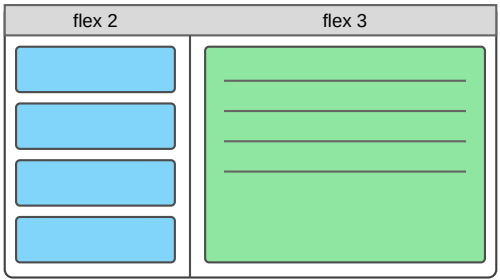
2.2 Minh hoạ bố cục theo không gian khả dụng



Mobile hẹp
$<$ 400 dp nội dung
1 cột



Mobile rộng
400–599 dp
2 cột



Tablet $\geq$ 600 dp
Master–Detail: danh sách | chi tiết

2.2 Code thực tế (1/3): chọn bố cục bằng LayoutBuilder

```dart
// breakpoints.dart
enum ScreenType { mobile, tablet }
static const double tablet = 600;
static ScreenType fromWidth(double width) =>
    width >= tablet ? ScreenType.tablet : ScreenType.mobile;

// home_screen.dart
Widget build(BuildContext context) => LayoutBuilder(
  builder: (context, constraints) {
    // Dựa vào không gian cha cấp, không suy đoán loại máy
    final type = Breakpoints.fromWidth(constraints.maxWidth);
    final detailVisible =
        type == ScreenType.mobile && _showDetail && ...;
    ...
    Expanded(
      child: type == ScreenType.tablet
          ? TabletLayout(notes: _notes, ...)
          : detailVisible
              ? NoteDetailScreen(note: selected, ...)
              : MobileLayout(notes: _notes, ...),
    ),
```

LayoutBuilder
Đo **`maxWidth`** của vùng cha cấp cho **`HomeScreen`**.

Breakpoint
`fromWidth()` đổi bề rộng thành **`ScreenType`** (ngưỡng **600 dp**).

Bảo toàn trạng thái
State (**`_notes`**, **`_selectedId`**) nằm **ngoài** LayoutBuilder nên đổi bố cục không mất dữ liệu.

2.2 Code thực tế (2/3): Master–Detail và lưới thích ứng

```dart
// tablet_layout.dart
Row(children: [
  Expanded(flex: 2, child: NoteList(...)),
  const VerticalDivider(width: 1),
  Expanded(flex: 3,
    child: selected == null
        ? const EmptyState(...)
        : NoteDetailView(note: selected!, ...)),
])

// responsive_helper.dart
int gridColumns(double availableWidth) {
  if (availableWidth >= Breakpoints.gridThreeColumns) return 3;
  if (availableWidth >= Breakpoints.gridTwoColumns) return 2;
  return 1;
}

// note_list.dart: đo riêng vùng danh sách
LayoutBuilder(builder: (context, constraints) {
  final padding = horizontalPadding(constraints.maxWidth);
  final columns = gridColumns(constraints.maxWidth - padding * 2);
  return GridView.builder(
    gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: columns, ...),
```

Adaptive
`Row` + `Expanded(flex: 2 / 3)` tạo bố cục **2 vùng** trên tablet.

Responsive
`gridColumns()` chia **1 / 2 / 3 cột** theo bề rộng nội dung (**400** và **900 dp**).

Đo đúng vùng
`NoteList` dùng LayoutBuilder **riêng** nên không phụ thuộc kích thước toàn màn hình.

2.2 Code thực tế (3/3): nhập liệu đa phương thức & trợ năng

```dart
// home_screen.dart: bàn phím
CallbackShortcuts(
  bindings: {
    const SingleActivator(LogicalKeyboardKey.keyN, control: true):
        () => _edit(),
  },
  child: Focus(autofocus: true, ...),
)

// note_card.dart: chuột
InkWell(
  mouseCursor: SystemMouseCursors.click,
  onTap: onOpen, ...
)

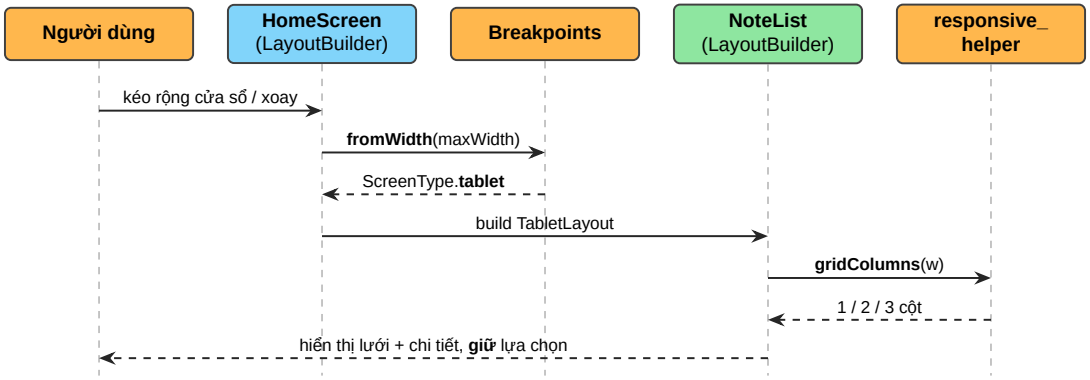
// note_list.dart: cảm ứng + cỡ chữ hệ thống
Dismissible(
  key: ValueKey(note.id),
  direction: DismissDirection.endToStart,
  onDismissed: (_) => onDelete(note), ...
)
// Chiều cao thẻ tăng theo cỡ chữ hệ thống
mainAxisExtent: 100 + scaler.scale(17) * 3 + ...,
// note_detail_view.dart: giới hạn bề rộng đọc
ConstrainedBox(
  constraints: const BoxConstraints(maxWidth: 720), ...)
```

Keyboard
Ctrl+N tạo ghi chú mới; **Ctrl+S** lưu trong màn hình sửa.

Mouse | **Touch**
`InkWell` cho hover và con trỏ; **`Dismissible`** cho **vuốt xoá**.

Trợ năng
`textScaler` làm thẻ cao hơn khi chữ lớn, **max-width** giữ dòng chữ dễ đọc.

2.3 Sơ đồ tuần tự: giao diện thích ứng khi đổi kích thước



2.3 Kịch bản demo

1. Mobile dọc ($<$ 400 dp)
Danh sách **1 cột**; Layout Debug Bar hiển thị kích thước và loại bố cục.

2. Xoay ngang / kéo rộng 400–599 dp
Lưới đổi sang **2 cột**, vẫn là bố cục Mobile.

3. Kéo rộng $\geq$ 600 dp
Chuyển sang **Master–Detail**: danh sách | chi tiết.

4. Chọn ghi chú rồi đổi kích thước
Lựa chọn và dữ liệu được **giữ nguyên**.

5. Chuột & bàn phím
Hover, **Ctrl+N**, **Ctrl+S**; vuốt trái để xoá rồi **Hoàn tác**.

6. Cỡ chữ lớn & Dark mode
Tăng cỡ chữ hệ thống (đến 2.0$\times$): không tràn chữ.

2.3 Kiểm thử tự động

Tệp kiểm thử	Nội dung kiểm tra
`breakpoints_test`	Đổi bố cục đúng tại **600 dp**; số cột tại 399 / 400 / 899 / 900; khoảng đệm theo bề rộng
`note_test`	Model **Note** bất biến; rút gọn xem trước 100 ký tự; dữ liệu mẫu độc lập
`widget_test`	Đổi kích thước **không mất lựa chọn**; thêm / sửa / xoá / hoàn tác; phím tắt; **không overflow** ở cỡ chữ 1.0–2.0×, theme sáng và tối
`check_layout_rules`	Công cụ kiểm tra tĩnh **12 quy tắc** thiết kế bố cục

3. Kết luận

- **Responsive** co giãn bố cục liên tục; **Adaptive** chọn bố cục riêng theo điều kiện. Ứng dụng thực tế **kết hợp cả hai**.
- Nên dùng **LayoutBuilder** để chọn bố cục theo **không gian khả dụng**, dùng **MediaQuery** cho thông tin hệ thống (cỡ chữ, hướng, vùng an toàn).
- Đặt **breakpoint** thành hằng số, giới hạn **max-width** nội dung, bảo toàn **trạng thái** khi đổi kích thước.
- **Adaptive Notes** đáp ứng yêu cầu: **Mobile / Tablet**, lưới **1–3 cột**, hỗ trợ **touch, mouse, keyboard**, trợ năng và kiểm thử tự động.

Xin cảm ơn
thầy và các bạn đã lắng nghe!

Hỏi & Đáp